

HallucinationAuto – Developer Reference

Tool-Calling Hallucination Project

2026

Contents

Overview	3
File layout	3
Inheritance chain	3
Quick start	3
schema.py – JSON Schema Tools	5
build_locked_schema(data, is_root=True, field_name="")	5
get_locked_schema_for_tool(tool_call_json_str)	5
create_cascade_schema(locked_schema) -> (chosen_leaf, unlocked_paths, schema)	5
get_random_focused_schema(locked_schema) -> (target, schema)	6
evaluate_hallucination(original_data, hallucinated_data, unlocked_paths) -> (spans, summary)	6
extract_value(obj, path) (<i>static</i>)	6
get_tool_names(tool_response_str) (<i>static</i>)	7
get_all_leaf_paths(schema), get_all_internal_paths(schema)	7
print_schema_tree(schema, ...)	7
singlehop.py – Single-Turn Generation	8
System prompts	8
type1_api(client, model, tool_response, user_prompt, howhow="smart", ...) -> (hall_dict, target, unlocked_paths)	8
type2_delete(tool_response, tagged_answer) -> (reduced_tr, deleted_paths, hallucination_spans, deletion_target)	9
type3_api(client, model, user_prompt, original_answer, system_tools, tool_call, ...) -> str	9
generate_type1_dataset(client, model, dataset, output_path, ...) -> list	10
generate_type2_dataset(dataset, output_path) -> list	10
add_missing_system(dataset, extra_datasets=None, output_path=None) -> list	11
generate_type3_dataset(client, model, dataset, output_path, ...) -> list	11
multihop.py – Multistep Injection and Pruning	13
Tagging helpers	13
_tag_row(row) (<i>static</i>) -> dict	13
tag_dataset(dataset, output_path=None, overwrite=False) -> list	13
Span detection	13

detect_type1_spans(item) -> (spans, plain_text)	13
detect_type2_spans(item) -> (spans, plain_text)	13
detect_type3_spans(item) -> (spans, plain_text)	14
Last-turn injection	14
_inject_last_turn(multistep, lookup, hall_type) -> (dataset, log)	14
generate_multistep_type1(multistep, type1_labelled, output_path) -> list	14
generate_multistep_type2(multistep, type2_data, output_path) -> list . .	14
generate_multistep_type3(multistep, type3_data, output_path) -> list . .	14
Pruning-based generation	15
extract_turn_as_row(dialogue, turn_k) (<i>static</i>) -> dict	15
prune_dialogue(dialogue, target_turn_k) (<i>static</i>) -> dict	15
inject_row_into_pruned(pruned_dialogue, hall_row, hall_type) -> dict . .	15
generate_type1_for_row(client, model, row, max_retries=3) (<i>async</i>) ->	
dict None	15
generate_type2_for_row(row) -> dict None	15
generate_type3_for_row(client, model, row, max_retries=3) (<i>async</i>) ->	
dict None	15
generate_pruned_multistep(client, model, multistep, output_dir,	
batch_size=5) (<i>async</i>)	16
export.py – Dataset Export	18
to_dataset(input_data, output_format="ragtruth", output_path=None)	18
_find_in_output(text, clean_output) (<i>static</i>)	18
showcase.py – Visualisation	19
showcase(client, model, dataset, n_examples=10, types=None, seed=42,	
timeout=120.0)	19
show_samples(data, n=3, seed=42, idx=None) (<i>static</i>)	19
Data Formats	21
Input dataset row (singlehop)	21
Multistep dialogue (input)	21
Pruned output row	21
Viewing Outputs	23
From a Jupyter notebook	23
From the terminal	23
Server Requirements	23

Overview

HallucinationAuto is a Python library for generating **tool-call hallucinations** in single-turn and multi-turn QA dialogues. It produces three types of hallucination, each targeting a different failure mode of LLM assistants:

Type	Name	LLM needed	What is corrupted
1	Schema-based corruption	Yes (vLLM + guided decoding)	Tool response values replaced with plausible fakes
2	Deletion-based overgeneration	No	Fields removed from tool response; original answer now over-claims
3	Tool-constrained overgeneration	Yes	One extra sentence added referencing an <i>unused</i> tool

File layout

```
last dub/
+-- hallucination_auto.py  <- entry point; imports all mixins
+-- schema.py              <- JSON schema tools (Type 1 backbone)
+-- singlehop.py           <- Type 1/2/3 API calls + dataset generation
+-- multihop.py            <- Multistep injection + pruning generation
+-- export.py              <- RAGTruth / CSV export
+-- showcase.py            <- HTML visualisation + terminal viewer
```

Inheritance chain

```
HallucinationAuto(
    SchemaMixin,      # schema.py
    SinglehopMixin,    # singlehop.py
    MultihopMixin,     # multihop.py
    ExportMixin,       # export.py
    ShowcaseMixin,     # showcase.py
)
```

Every method documented below is available on a single `HallucinationAuto` instance.

Quick start

```
import json, asyncio
from openai import AsyncOpenAI
import httpx, sys
sys.path.insert(0, "last dub")
```

```
from hallucination_auto import HallucinationAuto

ha = HallucinationAuto()

client = AsyncOpenAI(
    base_url="http://YOUR_SERVER:8000/v1",
    api_key="dummy",
    timeout=httpx.Timeout(300.0, connect=10.0),
)

dataset = json.load(open("dataset_v3_tagged_cleaned_sys.json"))

# Type 2 -- instant, no LLM
ha.generate_type2_dataset(dataset, "type2_output.json")

# Type 1 -- needs vLLM with outlines guided decoding
ha.generate_type1_dataset(client, "Qwen/Qwen2.5-14B-Instruct",
                           dataset, "type1_output.json")

# Type 3 -- needs LLM + 'system' column in dataset
ha.generate_type3_dataset(client, "Qwen/Qwen2.5-14B-Instruct",
                           dataset, "type3_output.json")

# Pruning-based multistep generation
multistep = json.load(open("last_dub/toolace_multistep_clean (1).json"))
asyncio.run(ha.generate_pruned_multistep(
    client, "Qwen/Qwen2.5-14B-Instruct",
    multistep, output_dir="pruning_gen/results",
))
```

schema.py – JSON Schema Tools

Class: SchemaMixin

Contains everything needed to build, lock, inspect, and selectively unlock JSON schemas for Type 1 guided decoding. The central idea: build a schema where every leaf value is pinned (`"enum": [original_value]`), then unlock a subset of fields so the LLM can hallucinate only those.

`build_locked_schema(data, is_root=True, field_name="")`

Recursively build a fully-locked JSON Schema from actual parsed data. Every leaf is pinned via `"enum"`. Arrays use `prefixItems` (one schema per element).

Parameters

- `data` – parsed Python object (dict, list, str, int, float, bool, None)
- `is_root` – internal; keep default
- `field_name` – internal; used to detect string-like ID fields

Returns dict – JSON-Schema-compatible dictionary

Example

```
import json
tool_resp = '{"name": "GetWeather", "results": {"temp": 72, "city": "NYC"}}'
data = json.loads(tool_resp)

locked = ha.build_locked_schema(data)
# locked["properties"]["results"]["properties"]["temp"]
# -> {"type": "integer", "enum": [72]}
```

`get_locked_schema_for_tool(tool_call_json_str)`

Convenience wrapper: parse a JSON string then call `build_locked_schema`.

```
locked = ha.get_locked_schema_for_tool(row["tool_response"])
```

`create_cascade_schema(locked_schema)` $\rightarrow$ (`chosen_leaf`, `unlocked_paths`, `schema`)

CASCADE (smart) masking – the recommended strategy for Type 1.

Picks a depth-weighted random leaf, then unlocks a contextually coherent region:

- **Array case:** leaf is inside an array element $\rightarrow$ unlocks the *whole element* so the LLM hallucinates a coherent row of data.
- **Flat case:** leaf is NOT in an array $\rightarrow$ unlocks $2\text{-ceil}(N/2)$ sibling fields under the same parent object.

Returns

- `chosen_leaf` – dot-path of the initially selected leaf
- `unlocked_paths` – all dot-paths that were actually unlocked
- `schema` – deep copy of `locked_schema` with the region unlocked

Example

```
locked = ha.get_locked_schema_for_tool(row["tool_response"])
target, paths, schema = ha.create_cascade_schema(locked)
print(target)      # e.g. "results.items.0.price"
print(paths)       # e.g. ["results.items.0.price", "results.items.0.title"]
```

`get_random_focused_schema(locked_schema) -> (target, schema)`

FOCUSED (dumb) masking – unlocks a random internal subtree entirely. Simpler than CASCADE; good for diverse corruption.

```
target, schema = ha.get_random_focused_schema(locked)
```

`evaluate_hallucination(original_data, hallucinated_data, unlocked_paths) -> (spans, summary)`

Compare original vs hallucinated tool responses; classify each change.

Labels

- "Evident Conflict" – a real value was replaced with a different value
- "Baseless Info" – a value was fabricated from null

Returns

- `spans` – list of dicts with `path`, `text`, `meta`, `label_type`, `implicit_true`, `due_to_null`
- `summary` – `{"evident_conflict": int, "baseless_info": int}`

Example

```
orig = json.loads(row["tool_response"])
hall = json.loads(row["hallucinated_tool_response"])
spans, summary = ha.evaluate_hallucination(orig, hall, unlocked_paths)
# summary -> {"evident_conflict": 2, "baseless_info": 0}
```

`extract_value(obj, path) (static)`

Extract a value from a nested dict/list using a dot-path. Numeric path segments are treated as list indices.

```
data = {"results": {"items": [{"price": 9.99}]}}
ha.extract_value(data, "results.items.0.price") # -> 9.99
```

get_tool_names(tool_response_str) (*static*)

Return tool/function names from a tool-response JSON string.

```
ha.get_tool_names('{"name":"GetWeather","results":{}}') # -> ["GetWeather"]
```

get_all_leaf_paths(schema), get_all_internal_paths(schema)

Traverse a schema and return dot-paths to leaf nodes or internal nodes. Used internally by masking strategies.

```
paths = ha.get_all_leaf_paths(locked)
# -> ["name", "results.temp", "results.humidity"]
```

print_schema_tree(schema, ...)

Print a human-readable tree of a JSON schema (for debugging).

```
ha.print_schema_tree(locked)
# `-- root (object)
#   +-- name (string) [LOCKED]
#   `-- results (object)
#       +-- temp (integer) [LOCKED]
#       `-- humidity (integer)
```

singlehop.py – Single-Turn Generation

Class: SinglehopMixin

Provides API call methods for all three hallucination types (single-turn), plus dataset generation loops (sync and async).

System prompts

Three class-level string constants override-able per call:

Constant	Used by
TYPE1_SYSTEM_PROMPT	type1_api
TYPE3_SYSTEM_PROMPT	type3_api – has {tools} placeholder
TESTING_TYPE_SYSTEM_PROMPT	testing_type_api

type1_api(client, model, tool_response, user_prompt, howhow="smart", ...) -> (hall_dict, target, unlocked_paths)

Generate a hallucinated tool response via **vLLM guided JSON decoding**.

Builds a partially-unlocked JSON schema, sends it as **guided_json** to vLLM, receives a JSON response where unlocked fields are hallucinated.

Parameters

- **client** – openai.OpenAI client pointed at your vLLM server
- **model** – model identifier, e.g. "Qwen/Qwen2.5-14B-Instruct"
- **tool_response** – original tool-response JSON string
- **user_prompt** – original user question (used as LLM input)
- **howhow** – "smart" (CASCADE) or "dumb" (FOCUSED)

Returns

- **hall_dict** – generated (corrupted) tool response as a parsed dict
- **target** – dot-path of the initially chosen leaf
- **unlocked_paths** – all dot-paths that were unlocked

Example

```
from openai import OpenAI
client = OpenAI(base_url="http://server:8000/v1", api_key="dummy")

hall_dict, target, paths = ha.type1_api(
    client, "Qwen/Qwen2.5-14B-Instruct",
    tool_response=row["tool_response"],
    user_prompt=row["user_prompt"],
    howhow="smart",
```

```
)
print(target)      # "results.temperature"
print(hall_dict)   # {"name": "GetWeather", "results": {"temperature": 68}}
```

Async variant: `type1_api_async(client, model, ...)` – same signature, requires `AsyncOpenAI`.

type2_delete(tool_response, tagged_answer) -> (reduced_tr, deleted_paths, hallucination_spans, deletion_target)

No LLM required. Delete fields from the tool response so the original answer now references data that no longer exists.

Strategy depends on data depth:

- `depth <= 1` (flat) -> delete 1-2 random leaf fields
- `depth >= 2` (nested/arrays) -> cascade-delete an entire array element

Parameters

- `tool_response` – original tool-response JSON string
- `tagged_answer` – assistant answer with `[field]value[/field]` markup (call `_tag_row` first if this column is missing)

Returns

- `reduced_tr` – JSON string with selected data removed
- `deleted_paths` – dot-paths of all deleted leaf values
- `hallucination_spans` – spans in the answer referencing deleted data
- `deletion_target` – the path(s) chosen for deletion

Example

```
reduced, paths, spans, target = ha.type2_delete(
    row["tool_response"], row["tagged_answer"]
)
print(paths)      # ["results.items.1.title", "results.items.1.price"]
print(len(spans)) # 2
```

type3_api(client, model, user_prompt, original_answer, system_tools, tool_call, ...) -> **str**

Generate one natural-sounding overgeneration sentence that subtly references a capability belonging to one of the **other** (unused) tools.

The used tool is excluded from the prompt so the LLM can only reference capabilities of *other* tools. The sentence sounds like a factual continuation, not a recommendation.

Parameters

- `system_tools` – raw `system` column containing the JSON tool definitions
- `tool_call` – the tool-call string (used to identify and exclude the used tool)

Returns `str` – a single overgeneration sentence

Example

```
comment = ha.type3_api(
    client, "Qwen/Qwen2.5-14B-Instruct",
    user_prompt=row["user_prompt"],
    original_answer=row["original_answer"],
    system_tools=row["system"],
    tool_call=row["tool_call"],
)
# -> "Historical price trends are also available for these indexes."
```

Async variant: `type3_api_async(client, model, ...)`.

`generate_type1_dataset(client, model, dataset, output_path, ...) -> list`

Generate Type 1 hallucinations for an entire dataset (**sync**, resumable).

Saves after every row. If `output_path` already exists with partial results, picks up from where it left off.

Each output row adds: `hallucinated_tool_response`, `hallucination_target`, `unlocked_paths`, `hallucination_type`, `eval_spans`, `eval_summary`.

Example

```
dataset = json.load(open("dataset_v3_tagged_cleaned_sys.json"))
result = ha.generate_type1_dataset(
    client, "Qwen/Qwen2.5-14B-Instruct",
    dataset, "corrupt/type1_output.json",
    howhow="smart",
)
print(len(result)) # e.g. 1387
```

Async variant: `generate_type1_dataset_async(client, model, dataset, output_path, batch_size=10, ...)` – processes `batch_size` rows concurrently with `asyncio.gather`.

`generate_type2_dataset(dataset, output_path) -> list`

Generate Type 2 hallucinations (**no LLM**, resumable).

Each output row adds: `reduced_tool_response`, `deleted_paths`, `deletion_target`, `hallucination_spans`, `hallucination_type`.

```
result = ha.generate_type2_dataset(dataset, "corrupt/type2_output.json")
```

```
add_missing_system(dataset, extra_datasets=None, output_path=None) -> list
```

Add a synthesised **system** column to rows that are missing one.

Type 3 generation requires a **system** field listing all available tools so the LLM knows which *other* tools it can reference. Some datasets (e.g. Glaive) do not include this column. This method builds a tool registry and backfills it.

Two-phase algorithm:

1. **Harvest real definitions** – scan **system** columns in **dataset** and any **extra_datasets** you supply; extract full tool definitions (name + description + parameters) from the JSON arrays embedded in those prompts.
2. **Fill gaps** – for tool names found in **tool_call** strings but not in the registry, build minimal definitions by parsing the call's argument names.

Every row without **system** receives a prompt listing **all** known tools. **_filter_system_tools** (called automatically by Type 3 generation) then removes the row's own tool, leaving the others for the LLM to reference.

Parameters

- **dataset** – list of row dicts to enrich
- **extra_datasets** – additional datasets to harvest real tool definitions from (e.g. your labelled type1/2/3 outputs which already have full system columns)
- **output_path** – if given, save the enriched dataset to this JSON path

Handles two tool_call formats:

```
# Function-style
'calculate_distance(origin="New York", destination="Los Angeles")'

# JSON-style (from Glaive)
'{"name": "get_stock_price", "arguments": "{\\"symbol\\": \\"AAPL\\"}"}'
```

Example

```
glaive = json.load(open("glaive_converted_toolcall_dataset.json"))
tagged = json.load(open("dataset_v3_tagged_cleaned_sys.json"))

# tagged has real tool definitions -> harvest them for glaive rows
enriched = ha.add_missing_system(glaive, extra_datasets=[tagged])
# Phase 1: 2527 real tool definitions harvested
# Phase 2: 2968 total tools in registry
# Phase 4: 14817/14817 rows enriched with synthetic system

# Now safe to run Type 3 on glaive
ha.generate_type3_dataset(client, model, enriched, "glaive_type3.json")
```

```
generate_type3_dataset(client, model, dataset, output_path, ...) -> list
```

Generate Type 3 hallucinations (**sync**, resumable). Skips rows without a **system** column.

Each output row adds: `overgenerated_answer`, `overgeneration_comment`, `used_tool`, `hallucination_type`.

```
result = ha.generate_type3_dataset(  
    client, "Qwen/Qwen2.5-14B-Instruct",  
    dataset, "corrupt/type3_output.json",  
)
```

Async variant: `generate_type3_dataset_async(...)`.

multihop.py – Multistep Injection and Pruning

Class: MultihopMixin

Two capabilities:

1. **Last-turn injection** – take pre-generated singlehop data and inject hallucinations into the last completed tool turn of each multistep dialogue.
2. **Pruning-based generation** – for a dialogue with N turns, generate hallucinated samples at every prefix depth from N down to 2.

Tagging helpers

`_tag_row(row) (static) -> dict`

Auto-tag an answer with `[field]value[/field]` markup by matching tool-response leaf values into the answer text. Longest-match-first, case-insensitive. Returns a shallow copy with `tagged_answer` added.

```
tagged = ha._tag_row(row)
# row["original_answer"] = "The temperature is 72 degF in New York."
# -> tagged["tagged_answer"] = "The temperature is [temp]72[/temp] degF in [city]New York[/city]"
```

`tag_dataset(dataset, output_path=None, overwrite=False) -> list`

Auto-tag every row in a dataset that is missing `tagged_answer`. Optionally saves result to `output_path`.

```
tagged_ds = ha.tag_dataset(dataset, "dataset_tagged.json")
```

Span detection

`detect_type1_spans(item) -> (spans, plain_text)`

Find char-level hallucinated spans for a Type 1 item. Uses a 4-tier strategy:

1. **Tag-based** – `[key]value[/key]` markup gives exact offsets
2. **Original text search** – search for original value string in plain text
3. **Corrupted text search** – search for hallucinated value in plain text
4. **Implicit** – mark entire answer if corruption found but nothing in text

```
spans, plain = ha.detect_type1_spans(row)
# spans[0] -> {"start": 45, "end": 52, "text": "40.748", "label_type": "Evident Conflict", ...}
```

`detect_type2_spans(item) -> (spans, plain_text)`

Find char-level spans for a Type 2 (deletion) item. Same 4-tier fallback, using `hallucination_spans` and `deleted_paths`.

```
detect_type3_spans(item) -> (spans, plain_text)
```

Find the hallucinated span for a Type 3 item. Trivially locates the `overgeneration_comment` substring in `overgenerated_answer`.

```
spans, plain = ha.detect_type3_spans(row)
# spans[0] -> {"start": 312, "end": 389, "text": "Historical trends...", ...}
```

Last-turn injection

```
_inject_last_turn(multistep, lookup, hall_type) -> (dataset, log)
```

Core injection function. Deep-copies multistep dialogues and corrupts only the last completed tool turn of each.

Parameters

- `multistep` – list of multistep dialogue dicts
- `lookup` – {`user_prompt`: hallucinated_row_dict}
- `hall_type` – "type1" | "type2" | "type3"

What it does

- **type1/type2**: replaces the last tool turn's JSON with the hallucinated version
- **type3**: leaves the tool turn unchanged; only replaces the assistant answer
- Wraps hallucinated spans in `<hall>...</hall>` tags (right-to-left insertion)
- Sets `hallucination_type` and `skipped` on each dialogue

Returns

- `dataset` – list of modified dialogue dicts
- `log` – list of per-dialogue info dicts (`dialogue_idx`, `n_spans`, `has_tags`, `skipped`)

```
lookup = {r["user_prompt"]: r for r in type2_data if r.get("reduced_tool_response")}
injected, log = ha._inject_last_turn(multistep, lookup, "type2")
```

```
generate_multistep_type1(multistep, type1_labelled, output_path) -> list
```

Inject Type 1 hallucinations into multistep dialogues (last turn only). `type1_labelled` must have `hallucinated_tool_response`, `hallucination_labels`, and `plain_answer` (produced by running `label_type1_dataset`).

```
type1_labelled = json.load(open("corrupt/type1_labelled.json"))
ha.generate_multistep_type1(multistep, type1_labelled, "final/multistep_type1.json")
```

```
generate_multistep_type2(multistep, type2_data, output_path) -> list
```

Inject Type 2. Spans computed on-the-fly via `detect_type2_spans`.

```
generate_multistep_type3(multistep, type3_data, output_path) -> list
```

Inject Type 3. Tool turn unchanged; only the assistant answer is replaced.

Pruning-based generation

extract_turn_as_row(dialogue, turn_k) (static) -> dict

Extract turn turn_k (0-indexed) from a multistep dialogue as a singlehop-compatible row dict.

The tool response is unwrapped from its JSON array [{name, results}] to a single object {name, results} for singlehop API compatibility.

Returns dict with: user_prompt, tool_response, tool_call, original_answer, system.

```
row = ha.extract_turn_as_row(dialogue, turn_k=1)
# row["user_prompt"] -> "Can you find zip code 10001?"
# row["tool_response"] -> '{"name": "GetZipCode", "results": {...}}'
```

prune_dialogue(dialogue, target_turn_k) (static) -> dict

Return a deep copy of dialogue truncated at target_turn_k (0-indexed). Keeps conversations[0 : assistant_answer_index + 1] for that turn. Updates analysis.completed_tool_turns and completed_tool_turn_count. Original dialogue is **not mutated**.

```
# 3-turn dialogue -> 2-turn dialogue (keep only turns 0 and 1)
pruned = ha.prune_dialogue(dialogue, target_turn_k=1)
print(pruned["analysis"]["completed_tool_turn_count"]) # 2
```

inject_row_into_pruned(pruned_dialogue, hall_row, hall_type) -> dict

Inject a single hallucinated singlehop row into the last turn of a pruned dialogue. Wraps _inject_last_turn with a single-item lookup.

```
injected = ha.inject_row_into_pruned(pruned, t2_row, "type2")
```

generate_type1_for_row(client, model, row, max_retries=3) (async) -> dict | None

Generate a Type 1 hallucination for a single singlehop row (async). Calls type1_api_async with retries. Returns enriched row with hallucinated_tool_response, hallucination_labels, plain_answer or None on failure.

generate_type2_for_row(row) -> dict | None

Generate a Type 2 hallucination for a single row (sync, no LLM). Auto-tags the answer if needed then calls type2_delete.

generate_type3_for_row(client, model, row, max_retries=3) (async) -> dict | None

Generate a Type 3 hallucination for a single row (async). Row must have a non-empty system field.

`generate_pruned_multistep(client, model, multistep, output_dir, batch_size=5)`
(async)

Main pruning orchestration. For each dialogue with N completed tool turns, generates hallucinated samples at depths N, N-1, ..., 2. Stops before depth 1 (single-hop – already have plenty of those).

```
3-turn dialogue -> [turn1, turn2, hall@turn3] depth=3
                   [turn1, hall@turn2]         depth=2
                   depth=1 -> STOP
```

Processes type2 synchronously (no LLM) and type1/type3 in async batches. Saves incrementally after each batch (resumable if interrupted).

Parameters

- `client` – AsyncOpenAI client
- `model` – model identifier
- `multistep` – list of multistep dialogue dicts
- `output_dir` – directory to save `pruned_type1/2/3.json`
- `batch_size` – concurrent async requests per batch for type1/type3

Returns (`all_type1`, `all_type2`, `all_type3`) – lists of injected dialogue dicts

Example

```
import asyncio, json
from openai import AsyncOpenAI
import httpx

client = AsyncOpenAI(
    base_url="http://server:8000/v1",
    api_key="dummy",
    timeout=httpx.Timeout(300.0, connect=10.0),
)

multistep = json.load(open("last_dub/toolace_multistep_clean (1).json"))

t1, t2, t3 = asyncio.run(ha.generate_pruned_multistep(
    client, "Qwen/Qwen2.5-14B-Instruct",
    multistep, output_dir="pruning_gen/results",
    batch_size=5,
))

print(len(t1), len(t2), len(t3)) # 192 192 192
```

Output schema per dialogue (in addition to original fields):

Field	Description
<code>hallucination_type</code>	"type1" / "type2" / "type3"
<code>skipped</code>	True if LLM returned same values (no real hallucination)
<code>_pruning_depth</code>	number of tool turns in this pruned sample
<code>_dialogue_id</code>	source dialogue identifier

Field	Description
<code>conversations[hall_turn]["hallucination_type"]</code>	marks the corrupted turn
<code>conversations[ass_turn]["value"]</code>	answer with <code><hall>...</hall></code> markup

export.py – Dataset Export

Class: ExportMixin

Convert generated hallucination data to RAGTruth CSV or JSON format.

to_dataset(input_data, output_format="ragtruth", output_path=None)

Convert generated data to a structured dataset file.

Parameters

- `input_data` – path to a JSON file from `generate_type*_dataset`, or a list of dicts
- `output_format` – "ragtruth" (CSV) or "json" (list of dicts)
- `output_path` – override the default output path

RAGTruth CSV fields: id, query, context, output, task_type, quality, model, temperature, hallucination_labels, hallucination_labels_processed, input_str

Example

```
# -> saves type1_output_ragtruth.csv alongside the input file
ha.to_dataset("corrupt/type1_output.json", output_format="ragtruth")

# -> returns list in memory
rows = ha.to_dataset(my_list, output_format="json")

# -> saves to specific path
ha.to_dataset("corrupt/type1_output.json",
              output_format="json",
              output_path="final/type1_converted.json")
```

_find_in_output(text, clean_output) (static)

Locate text inside clean_output using 7 progressively fuzzier strategies:

1. Exact substring match
2. Integer with thousands comma (300000 -> 300,000)
3. Float with thousands comma (12345.67 -> 12,345.67)
4. Case-insensitive search
5. ISO date prefix (2025-01-15T10:00:00Z -> 2025-01-15)
6. Short alphanumeric word-boundary regex
7. Short digit word-boundary regex

Returns (start, end, matched_text) or (-1, -1, text) if unmatched.

showcase.py – Visualisation

Class: ShowcaseMixin

Two visualisation tools: HTML cards for Jupyter notebooks and a terminal viewer.

```
showcase(client, model, dataset, n_examples=10, types=None, seed=42, timeout=120.0)
```

Generate and display a rich **HTML showcase** of all three hallucination types in a Jupyter notebook cell.

Produces `n_examples` cards per type with colour-coded layout:

- **Red** – Type 1: original vs hallucinated tool response side-by-side
- **Blue** – Type 2: original vs reduced tool response; answer with highlighted spans
- **Green** – Type 3: original answer vs answer with overgeneration sentence

Parameters

- `types` – [1, 2, 3] by default; pass e.g. [2] to show only Type 2
- `seed` – random seed for reproducible sampling

Requirements

Each dataset row must have: `user_prompt`, `tool_response`, `tool_call`, `original_answer`, `tagged_answer`. For Type 3 also `system`. Type 1 and 3 require a live vLLM server.

Example

```
from openai import OpenAI
client = OpenAI(base_url="http://server:8000/v1", api_key="dummy")

ha.showcase(client, "Qwen/Qwen2.5-14B-Instruct",
            dataset, n_examples=5, types=[2])
```

```
show_samples(data, n=3, seed=42, idx=None) (static)
```

Print `n` random (or one specific) dialogues from a pruned dataset. Works in any terminal or Jupyter cell – plain text with ANSI colour. Hallucinated turns are marked * **HALLUCINATED** and `<hall>` spans are highlighted in red.

Parameters

- `data` – list of dialogue dicts (from `pruned_type*.json`)
- `n` – number of random samples (ignored when `idx` is set)
- `seed` – random seed
- `idx` – if given, show only this single dialogue index

Example

```
data = json.load(open("last dub/pruneddataset/pruned_type2.json"))

ha.show_samples(data, n=3)           # 3 random dialogues
ha.show_samples(data, idx=42)        # specific index
ha.show_samples(data, n=5, seed=99)  # different random picks
```

Data Formats

Input dataset row (singlehop)

```
{
  "user_prompt": "What is the weather in Paris?",
  "tool_call": "GetWeather(city=\"Paris\")",
  "tool_response": "{\"name\": \"GetWeather\", \"results\": {\"temp\": 18}}",
  "original_answer": "The current temperature in Paris is 18 degC.",
  "tagged_answer": "The current temperature in Paris is [temp]18[/temp] degC.",
  "system": "[{\"name\": \"GetWeather\", ...}, {\"name\": \"GetForecast\", ...}]"
```

Multistep dialogue (input)

```
{
  "system": "You are an expert...",
  "conversations": [
    {"from": "user", "value": "Find quotes about inspiration."},
    {"from": "assistant", "value": "[QuotesByKeywords(word=\"inspiration\")]"},
    {"from": "tool", "value": "[{\"name\": \"QuotesByKeywords\", \"results\": {...}}]"},
    {"from": "assistant", "value": "Here are some quotes: ..."}
  ],
  "analysis": {
    "dialogue_id": "toolace_00001",
    "completed_tool_turn_count": 2,
    "completed_tool_turns": [
      {
        "turn_number": 1,
        "user_index": 0,
        "assistant_tool_call_indices": [1],
        "tool_result_indices": [2],
        "assistant_answer_index": 3
      }
    ]
  }
}
```

Pruned output row

Same as multistep dialogue, plus:

```
{
  "hallucination_type": "type2",
  "skipped": false,
  "_pruning_depth": 2,
  "_dialogue_id": "toolace_00001",
  "conversations": [
```

```
...,
{
  "from": "tool",
  "value": "[{\"name\": \"QuotesByKeywords\", \"results\": {\"quotes\": [...]}}]",
  "hallucination_type": "type2",
  "hallucination_target": ["results.quotes.1"]
},
{
  "from": "assistant",
  "value": "Here are some quotes:\n\n1. \"...\"\n2. <hall>\"Be the change...\" -- Gandhi</h"
}
]
```

Viewing Outputs

From a Jupyter notebook

```
import json, sys
sys.path.insert(0, "last dub")
from hallucination_auto import HallucinationAuto
ha = HallucinationAuto()

data = json.load(open("last dub/pruneddataset/pruned_type2.json"))
ha.show_samples(data, n=3)
```

From the terminal

```
python3 scripts/show_samples.py --type type2 -n 3
python3 scripts/show_samples.py --type type1 --idx 42
python3 scripts/show_samples.py --type type3 -n 5 --seed 99
```

Server Requirements

Type 1 and Type 3 generation require a **vLLM** server with:

```
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-14B-Instruct \
  --dtype half \
  --tensor-parallel-size 4 \
  --guided-decoding-backend outlines # required for Type 1
```

Recommended timeout for the client:

```
from openai import AsyncOpenAI
import httpx

client = AsyncOpenAI(
    base_url="http://SERVER_IP:8000/v1",
    api_key="dummy",
    timeout=httpx.Timeout(300.0, connect=10.0),
)
```

Test connectivity:

```
curl -s http://SERVER_IP:8000/v1/models | python3 -m json.tool
```